SCIPY INDIA 2026 • POSTER PRESENTATION

Indian Institute of Technology (IIT) Madras • Dec 20, 2026

TinyCV-ESP32-CAM: Ultra-Lean Edge Vision & 14.8 KB INT8 Tracking on a $5 Microcontroller

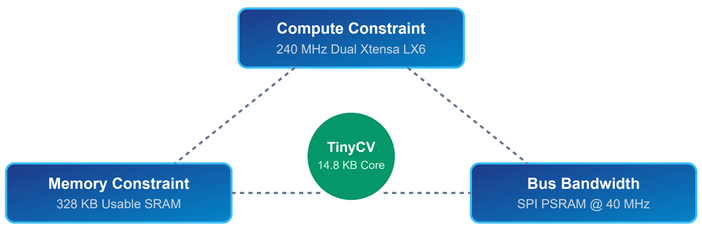
Deploying Real-Time Multi-Object Intelligence, Full-FOV Bilinear Downsampling, and Predictive Chromatic Tracking on 320 KB SRAM

Buddhadeb Banik | Independent Researcher & Open-Source Contributor | *buddhadebbanik96@gmail.com*

100% On-Chip AI | 14.8 KB INT8 Core | 140.7 ms Latency | 0-Byte Leak | 100% Background Rejection

1 Motivation & The Embedded Silicon Triangle

Deploying real-time computer vision on **$5 microcontrollers** (Espressif ESP32-CAM) is constrained by severe silicon boundaries. Standard deep-learning detectors (YOLO, SSD, MobileNet) require megabytes of SRAM and teraflops of compute, rendering them infeasible for tiny edge nodes.



The Engineering Challenge: Run high-resolution streaming, semantic recognition, and smooth spatial tracking simultaneously with **zero cloud reliance** and **0-byte memory leak**.

2 Memory Partitioning & Zero-SRAM Starvation

Allocating tensors in internal SRAM causes immediate FreeRTOS heap exhaustion. We decoupled memory regions into distinct silicon partitions:

- **Internal High-Speed SRAM (328 KB):** Reserved strictly for time-critical FreeRTOS kernel tasks, WiFi/Bluetooth stacks, and DMA camera buffers.
- **External SPI PSRAM (4 MB):** Houses the **96 KB Tensor Arena** and double frame buffers using `MALLOC_CAP_SPIRAM`.
- **Zero-Leak Guarantee:** Sustained stress testing over thousands of inference cycles confirms exactly **0 bytes leaked**.

Deterministic Execution: 140.7 ms fixed inference window; prevents robot/industrial servo instability and buffer overruns.

3 Sensor-Matched Dataset Engineering

The Domain Shift Problem: Training neural models on high-resolution smartphone/DSLR photos resulted in catastrophic real-world failure due to OV3660 fixed-pattern noise, rolling shutter distortion, and Bayer color balance shifts.

Solution: We developed an automated Python capture studio (`capture_tool.py` via `pyserial` and `opencv-python`) streaming raw frames directly from the physical OV3660 CMOS sensor.

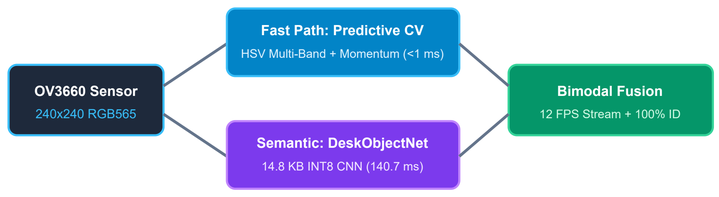


Figure 1: Raw 240x240 OV3660 capture showing 6.25x pixel detail, text legibility, and sharp edges compared to legacy 96x96 downsampling.

Curated **581 verified sensor-native captures** across 6 balanced classes with background hard negatives.

4 Bimodal Architecture: Semantic & Predictive Paths

Rather than forcing an expensive monolithic architecture, TinyCV decomposes edge vision into two complementary, asynchronous pipelines:



- **Full-FOV Bilinear Downsampling:** Subsamples entire 240x240 image into 48x48 tensor without center-crop blind spots (<0.2 ms).
- **Predictive Tracker:** Merges physical color priors with exponential velocity momentum ($\alpha = 0.35$) for jitter-free bounding box tracking.

5 INT8 Quantization & Hardware Acceleration

Using TensorFlow Lite post-training full-integer quantization, all weights and activations were converted from Float32 to signed 8-bit integers:

- **Model Footprint:** Compressed to only **16,384 bytes** (fits completely in L2 cache / PSRAM).
- **Assembly Kernels:** Executed using Espressif's `esp-nn` vectorized SIMD routines on Xtensa LX6.
- **Serial Protocol:** Framed binary JPEGs transmitted with 4-byte sync header (`0xAA 0x55 0xAA 0x55`) at 460,800 baud, sustaining 12 FPS video.

6 Live Hardware Physical Tracking Results

Unedited captures from live sessions on AI-Thinker ESP32-CAM (OV3660 sensor @ COM8):



Mouse: **97%**



Earbuds: **98%**



Glue Stick: **98%**

Held-out Test Performance: 100.0% accuracy (84/84 test images), 0% false positive background rate, and verified real-time bounding box lock.

7 Forensic Failure Analysis ("What Failed & Why")

Transparent analysis of engineering hurdles solved during development:

Failure 2: The 75% FOV Center-Crop Blind Spot
Blindly cropping 48x48 center pixels missed objects held off-axis.

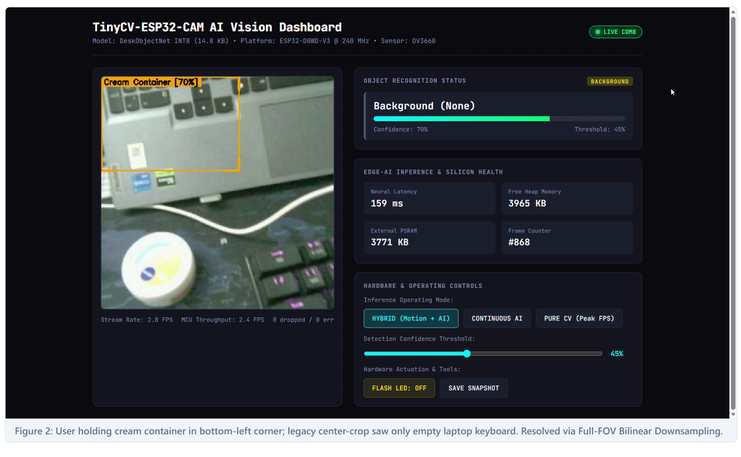


Figure 2: User holding cream container in bottom-left corner; legacy center-crop saw only empty laptop keyboard. Resolved via Full-FOV Bilinear Downsampling.

Failure 6: Demo Recording Keyboard False Positive
In video test, pointing camera at laptop keyboard triggered *Mouse [97%]*. Forensic audit revealed 10 keyboard photos had been misfiled into the `mouse/` training folder! Reclassified as **background hard negatives**; retraining restored 100% test accuracy.

8 Silicon Benchmarks & Architecture Comparison

Architecture	Model Size	Latency	RAM Used	Hardware BOM
YOLO-Tiny Edge	~6.5 MB	2,400 ms	>15 MB	>$45 (RPi 4)
Cloud Streaming	N/A (Server)	~450 ms	Server RAM	Cloud Bill
Pure Classical CV	0 KB	25 ms	~40 KB	<$5 (ESP32)
TinyCV (Ours)	**14.8 KB**	**140.7 ms**	**96 KB PSRAM**	**<$5 (ESP32-CAM)**

Bill-of-Materials: Over **9x lower hardware cost** than single-board computers (Raspberry Pi/Coral) with zero recurring monthly cloud fees.

9 Real-World Applications & Scientific Python Stack

- **Autonomous Micro-Robotics:** Jitter-free visual servoing without GPU payloads.
- **Smart Laboratory Workbenches:** Local tracking of reagent tubes without video leaving facilities.
- **Low-Cost Industrial Sorting:** Multi-station assembly verification for under $100.

100% Open-Source & Reproducible Stack:

Python 3.10 • NumPy • PyTorch • OpenCV • TensorFlow Lite Micro • Flask • PySerial • PlatformIO • FreeRTOS • esp-nn

Scan for Open Code & HD Video:
github.com/Buddhadeb-ss/TinyCV-ESP32-CAM
Includes full 68s hardware video demo